

基于 Flush + Reload 的 DES 算法 Cache 计时攻击

程志炜^a, 陈财森^b, 邱雪欢^a

(陆军装甲兵学院 a. 信息工程系; b. 科研学术处, 北京 100072)

摘要: 在数据加密标准(DES)算法上使用 Flush + Reload 方法的 Cache 计时攻击, 存在难以确定 S 盒元素行偏移位的问题。为此, 利用 S 盒在 Cache 中会发生不对齐分布的特征, 提出一种偏移位确定方法。采用 clflush 指令刷新 Cache 地址行, 以驱逐 Cache 中 DES 算法的 S 盒信息, 确定 S 盒元素的位置信息。利用密码算法在查询 S 盒时泄露的 Cache 计时信息获取明文与密钥异或后的中间态值, 并通过该值和已知明文的相关性恢复密钥。实验结果表明, 该方法能够有效确定 S 盒元素的行偏移位, 提高针对 DES 算法 Cache 计时攻击的效率。

关键词: 数据加密标准算法; Flush + Reload 攻击; Cache 计时攻击; S 盒; 行偏移

中文引用格式: 程志炜, 陈财森, 邱雪欢. 基于 Flush + Reload 的 DES 算法 Cache 计时攻击 [J]. 计算机工程, 2018, 44(12): 163-167.

英文引用格式: CHENG Zhiwei, CHEN Caisen, QIU Xuehuan. Cache timing attack on DES algorithm based on Flush + Reload [J]. Computer Engineering, 2018, 44(12): 163-167.

Cache Timing Attack on DES Algorithm Based on Flush + Reload

CHENG Zhiwei^a, CHEN Caisen^b, QIU Xuehuan^a

(a. Department of Information Engineering; b. Department of Academic Research,
Academy of Army Armored Force, Beijing 100072, China)

【Abstract】 Aiming at the problem that it is difficult to determine the row offset of the S-box elements for the Cache timing attack using the Flush + Reload method on Data Encryption Standard(DES) algorithm, a method is proposed to determine the row offset by using the property that S-box will be misaligned in the Cache. To evict S-box information of the DES algorithm from the Cache, the attack uses the clflush instruction to flush cache line. When the DES algorithm queries the S-box, it leaks the information of Cache timing. The intermediate state value which equals the plaintext and key XOR is recovered by utilizing this leaked information. Utilizing the value and the known plaintext, the key of DES is recovered. Experimental results show that this method can determine the row offset of S-box elements and improve the efficiency of Cache timing attack against DES algorithm.

【Key words】 Data Encryption Standard(DES) algorithm; Flush + Reload attack; Cache timing attack; S-box; row offset

DOI: 10.19678/j.issn.1000-3428.0049471

0 概述

传统密码算法攻击方法主要是对密码算法进行强力攻击以及利用算法设计缺陷来攻击, 分组密码算法在数学上往往是足够安全的, 因此, 利用传统攻击方法复杂度非常高, 效率极低。20 世纪末, 旁路攻击概念的出现颠覆了传统密码界的攻击观念, 攻击者利用密码算法在运行时泄露的旁路信息来破解密码算法。文献[1]提出访问驱动 Cache 分析, 随后文献[2]借鉴这种思想, 实现了在特定环境下针对高级加密标准(Advanced Encryption Standard, AES)算法的访问驱动 Cache 计时分析。传统的 Cache 分析^[3]

主要利用间谍进程(Spy Process, SP)清空 Cache, 当密码算法运行时从 Cache 中将 SP 访问过的 Cache 行数据替换出去, 然后再次启动间谍进程 SP, 利用 Cache 的命中与失效信息来判断密码算法访问过的 Cache 行信息, 通过 S 盒索引值与 Cache 行的对应关系确定访问的 S 盒元素, 这种方法在实现过程中比较困难, 并且效率不高。文献[4]提出了 Flush + Reload 计时攻击方法, 与传统的 Cache 计时分析不同, 这种攻击方法不需要清空整个 Cache, 只需要刷新特定的 Cache 行就能实现攻击。该方法需要映射密码算法共享执行库到间谍 SP 进程中, SP 使用 clflush 指令刷新指定地址行, 运行密码算法后, 再次

基金项目: 国家自然科学基金(61402528)。

作者简介: 程志炜(1993—), 男, 硕士研究生, 主研方向为信息安全; 陈财森, 讲师、博士; 邱雪欢, 硕士研究生。

收稿日期: 2017-11-28 修回日期: 2017-12-31 E-mail: cheng.zw@mail.scut.edu.cn

运行 SP 进程来判断刷新过的地址是否被访问过。与传统的访问驱动 Cache 计时攻击相比, Flush + Reload 攻击更容易实现, 并且效率更高。该攻击方法能够确定密码进程使用了哪些具体指令和访问了哪些具体数据, 基于 Flush + Reload 方法在 AES 算法分析上具有良好的效果^[5-4]。文献 [7] 利用这种攻击方法探测出运行在主机上的不同加密算法库, 文献 [8] 则利用这种攻击方法推断出键盘的输入信息。

Flush + Reload 方法对对称密钥算法的 S 盒具有良好的攻击效果, 但是因为 clflush 指令会刷新整个 Cache 行, 而不是 Cache 行中的某一个部分, 所以基于 clflush 指令的 Flush + Reload 计时攻击方法精度只能定位到 Cache 行, 因此, 该方法难以确定 S 盒元素在 Cache 行中的偏移位。本文以数据加密标准 (Data Encryption Standard, DES) 算法为研究对象, 介绍 DES 算法和 Flush + Reload 访问驱动攻击方法的实现原理, 以及 Cache 的结构与信息泄露模型, 分析 Flush + Reload 访问驱动 Cache 计时攻击方法存在的不足。在此基础上, 利用 S 盒在 Cache 中会发生不对齐分布的特征对 DES 密码算法进行攻击。

1 DES 算法计时攻击原理

1.1 DES 算法原理

DES 算法是一种分组对称加密算法, 以 64 位为分组, 密钥长度为 64 位, 参与运算的密钥有 56 位, 其余 8 位属于密钥的校验位, DES 算法基本构造原件主要包括 IP 置换及逆置换 IP^{-1} 、 f 函数、 E 扩展置换、S 盒代替和密钥编排^[9]。算法加密流程主要如图 1 所示。

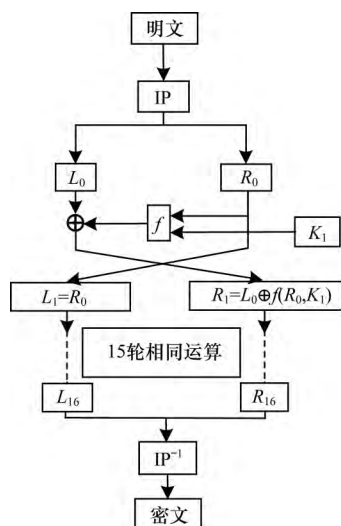


图 1 DES 算法加密流程

DES 算法加密的运算步骤如下:

1) 将 64 位明文进行 IP 置换, 输出为左 32 位 L_0 和右 32 位 R_0 。

2) 将上一轮右 32 位 R_{i-1} 和该轮密钥 K_i 输入函数 f 中, 输出 32 位用来加密左半部分 L_{i-1} , f 函数数学表示为:

$$f: \{0,1\}^{32} \times \{0,1\}^{48} \rightarrow \{0,1\}^{32}$$

f 函数是 DES 算法的关键, 也是计时攻击的对象。 f 函数包括扩展函数 $E()$ 、模 2 加、S 盒置换函数^[10], 其原理如图 2 所示。

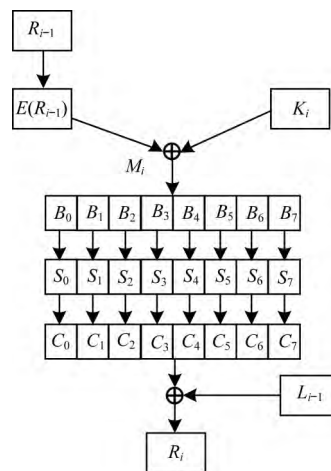


图 2 f 函数原理示意图

在图 2 中, R_{i-1} 是 32 位, 经过扩展函数 E 扩展, 输出 48 位中间值 $E(R_{i-1})$ 与密钥 K_i 进行异或后得到 M_i 作为 S 盒的输入, S 盒置换函数是 f 函数的关键, 是一种用于抵抗差分密码分析的非线性映射表。 f 函数有 8 个 S 盒, 每个 S 盒的输入是 6 位输出是 4 位, 整个 S 盒的输出 32 位值与 L_{i-1} 进行异或得到 R_i 。

攻击主要针对 S 盒替换, 假设输入已知明文 P 经过置换后的右 32 位为 R_i , 算法第一轮密钥为 K_1 , E 为扩展函数。针对第一轮加密, 通过基于访问驱动的 Cache 计时攻击获得查询 S 盒的信息, 得到输入 S 盒的值 M_1 。依据 DES 算法的原理有: $E(R_1) \oplus K_1 = M_1$ 。通过已知明文, 从数学上推导出 $E(R_1)$ 的值^[11], 结合获得的值 M_1 , 最终推导出 K_1 的值, 最后通过 DES 算法密钥扩展关系逆推得到完整的密钥。

3) 在第一轮加密后, 进行 15 轮同样的操作, 最后一轮输出经过 IP^{-1} 置换, IP^{-1} 与 IP 操作完全相反, 最后得到密文。上一轮的输出与下一轮的输入关系为:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

密钥 K_i 是从原始的 56 位密钥中得到 16 个轮密钥 K_i , 其中每个轮密钥 K_i 都是 48 位。

1.2 Cache 信息泄露原理

Cache 是位于 CPU 和主存之间的静态存储器, 存取速度仅次于 CPU 寄存器, 用以解决 CPU 与主存之间速度不匹配的问题^[12]。Cache 只保存内存中的一些数据, 在 Cache 中存有的数据同样也存储在内存中, 当 CPU 访问数据 D 时, 如果数据 D 存在 Cache 中, 称为 Cache 命中, 并取出数据 D 送到 CPU 中; 如果 D 不在 Cache 中, 就会发生 Cache 失效, 并从内存中读取数据, 同时将数据 D 载入 Cache 中^[13]。由于程序的局部性特点, 在 Cache 失效时, 会将内存中的整块数据调入 Cache 中, 目的是为了提提高命中率, 当

Cache 已经填满时,就会使用替换策略,将 Cache 中的数据逐出。

由于 Cache 的存取速度远大于内存,因此从 Cache 中读取数据的时间小于从内存中读取的时间。密码算法运行时泄露的 Cache 的命中和 Cache 失效信息,能够被用来获取密码加密时使用的 S 盒信息。RDTSC 指令^[4]能够用于获取 CPU 上的执行周期,利用 RDTSC 指令可区分 Cache 命中和失效信息。在计算执行周期时,不考虑 RDTSC 指令带来的噪声影响,Cache 命中和失效区别信息如图 3 所示。

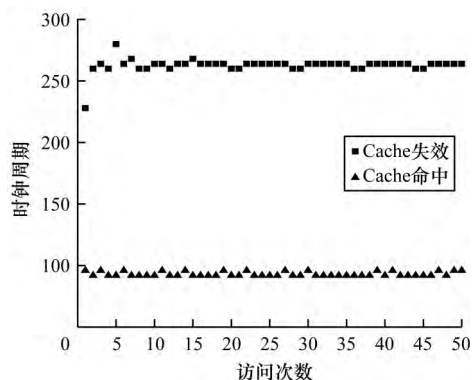


图3 Cache命中和失效的区别信息

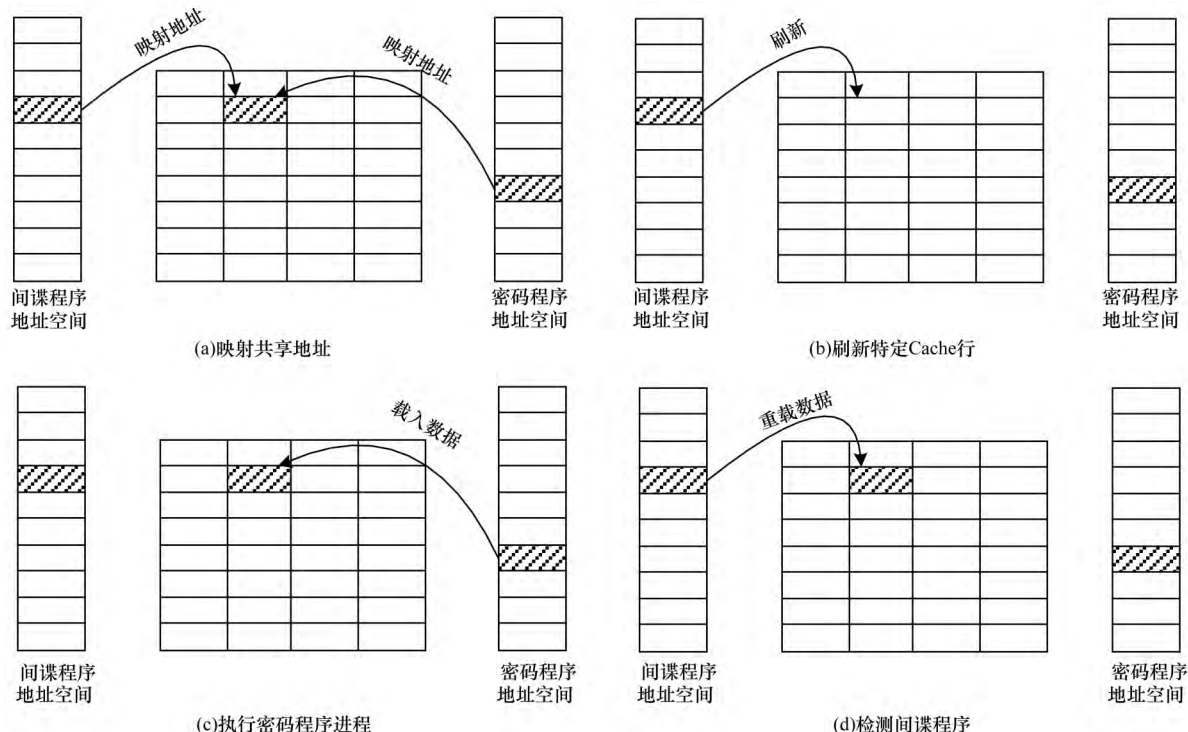


图4 Flush + Reload 攻击原理示意图

Flush + Reload 算法执行步骤如下:

- 1) 将二进制(共享库)映射入共享地址空间(进入Cache中)。
- 2) 从Cache中使用 clflush 指令刷新一个共享的Cache行。
- 3) 调用密码程序进程执行。

使用 clflush 指令刷新某一 Cache 行,如果该 Cache 行之前的存有加密时使用的 S 盒,则会发生 Cache 失效。Cache 和主存之间传递数据是以“块”为单位的,当发生 Cache 失效时,从内存中读取数据的时间明显大于从 Cache 中读取数据的时间,利用 Cache 泄露的旁路信息,能够利用获取查询的 S 盒信息。

1.3 Flush + Reload 计时攻击分析

文献[4]提出基于 clflush 指令实现更高效的访问驱动攻击,其利用 clflush 指令把数据从 Cache 中驱逐到主存中,当密码算法执行一小部分指令时,重新检测被驱逐的数据是否再次加载到 Cache 中。假设 Cache 有 8 组 4 路,Flush + Reload 攻击原理如图 4 所示。其中:目标地址(共享)映射到密码程序和间谍程序中,如图 4(a)所示;间谍程序刷新共享的目标地址,共享目标地址从 Cache 中被逐出,如图 4(b)所示;然后运行密码进程,发生 Cache 失效,重新从主存中读取数据并载入 Cache 中,如图 4(c)所示;最后间谍程序通过利用 Cache 命中信息来检测前面刷新的目标地址是否重新被载入 Cache 中,如图 4(d)所示。

- 4) 检测第 2 个步骤中被刷新的 Cache 行是否被密码程序载入到 Cache。

在第 1 个步骤中,使用 Linux 系统调用函数 mmap 来进行映射共享库;在第 2 个步骤中,使用 clflush 指令从 Cache 中驱逐的数据包括共享的 LLC (Last-Level Cache)^[3];在访问驱动攻击中,Flush + Reload 攻击是一

种更加细粒度的攻击^[8],能够确定受害程序使用了哪些具体指令和访问了哪些具体数据。

2 基于 Cache 不对齐特征的攻击实现

针对 DES 算法的 8 个 S 盒,根据 S 盒的结构,实现 DES 算法的 S 盒,每个元素占 4 Byte,加载一个 4 行 16 列的 S 盒需要 256 Byte。在 Intel i7-4720HQ 处理器中,每个 Cache 行大小为 64 Byte,clflush 指令刷新大小为 64 Byte。如果 S 盒查找表在 Cache 中是对齐分布,则一个 S 盒需要 4 个 Cache 行存储;如果不对齐,则一个 S 盒占用了 5 个 Cache 行,如图 5 所示。

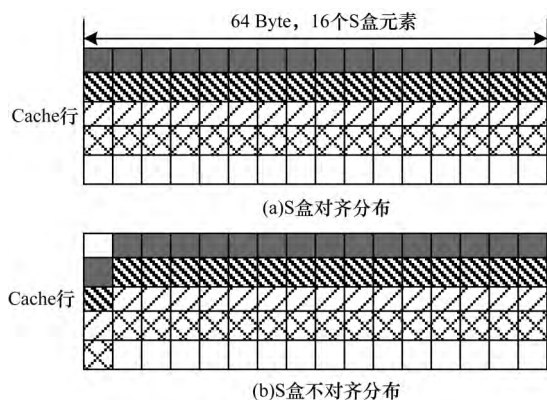


图 5 S 盒在 Cache 中的分布特征

使用 Linux 系统调用函数 mmap 映射 DES 算法加密共享库到间谍进程 SP 中,运行加密算法进程后刷新指定 Cache 行,因为一次会刷新 64 Byte,所以如果 S 盒在 Cache 中对齐分布,使用该方法只能确定访问的 S 盒所在行。DES 算法有 8 个 S 盒,只对第一轮加密进行攻击,使用 Flush + Reload 方法进行一轮攻击,能够确定加密时使用的 S 盒所在的 Cache 行。

DES 在算法查找 S 盒时,使用 6 个比特位来确定查找的元素,首尾两位确定查找行,中间 4 位用来确定列。经过一轮攻击,确定了行位置,能将 DES 算法的密钥搜索空间从 2^{56} 降到 2^{32} 。

在 DES 算法实际运算环境中,S 盒查找表在 Cache 中不都是对齐分布,根据这种特征,本文提出一种新的方法,用于准确获取查询的 S 盒元素在 Cache 行中的准确偏移位。

在第一轮攻击后,确认查询的 S 盒元素所在的 Cache 行。设偏移量为 0 的地址为 $V[0]$,将刷新地址右偏移 $4 \times i$ Byte 的地址为 $V[i]$ (其中 $i \in [0, 15]$ 且 i 为整数),每次偏移量刚好是 S 盒一个元素的大小。当发生 Cache 不对齐时,刷新的地址和 S 盒查询的地址在同一 Cache 行时,间谍进程第 2 次访问刷新的地址时会发生 Cache 命中,反之,不在同一 Cache 行时,就会发生 Cache 失效。对算法攻击 T 次,S 盒查找表在 Cache 中会发生多次不对齐,设间谍进程发生 Cache 失效的次数为 n ,如果刷新地址不是查询使用的 S 盒元素,则有 $n > 0$;如果刷新的地

址刚好是查询使用的 S 盒元素,则有 $n = 0$ 。算法实现过程如下所示。

```

输入 明文  $p$ ; 密钥  $k$ ; Cache 行首地址  $V[0]$ 
输出 行偏移地址失效次数
1. DES( $p, k$ );
2. int * addr =  $V[0]$ ;
3. int count [6];
4. for( int i = 0; i < 16; i++)
5. {
6. count[i] = 0;
7. for( int j = 0; j < N; j++)
8. {
9. clflush( addr + i);
10. DES(  $p, k$ );
11. int time = rdtsc();
12. maccess( addr);
13. int delta = rdtsc() - time;
14. if( delta > MAX_CACHE_HIT_CYCLES)
15. count[i]++;
16. }
17. printf( "%d\n", count[i]);
18. }

```

在准确获取查询 S 盒的所有索引值 M_1 后,首先通过已知明文扩展 $E(R_1)$,根据 $K_1 = E(R_1) \oplus M_1$ 得到密钥扩展的第一个密钥;然后根据数学关系推算推出 DES 算法的密钥;最后使用密钥对密文进行解密,以验证出密钥的正确性。

3 实验与结果分析

攻击实验在 Ubuntu Kylin 16.04 的操作系统下进行,分析目标为在 Ubuntu 中编写的 DES 算法密码库 des.so,密钥为随机生成密钥。CPU 为 4 核 Intel i7-4720HQ,每个核的 L1 Cache 大小为 32 KB,相联度为 8 路,Cache 行大小为 64 Byte,clflush 指令刷新大小 64 Byte。

在实验环境中,在使用已知固定明文情况下,首先确定使用的 S 盒行位置,实验过程中进行了 4 次攻击,实际上只需要一次攻击就能确定所在行,将密钥的搜索空间降为 2^{32} ,针对第一个 S 盒的实验结果如图 6 所示,确定使用的 S 盒为第 4 行。

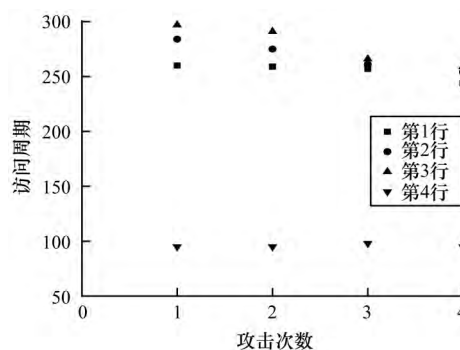


图 6 S 盒的行位置

在确认 S 盒行之后,需要利用 S 盒在 Cache 中发生的不对齐现象来确定偏移位,本文使用上述方法进行 500 次攻击实验,采集每次攻击的偏移位地址发生 Cache 失效的次数,针对第一个 S 盒的实验结果如图 7 所示。

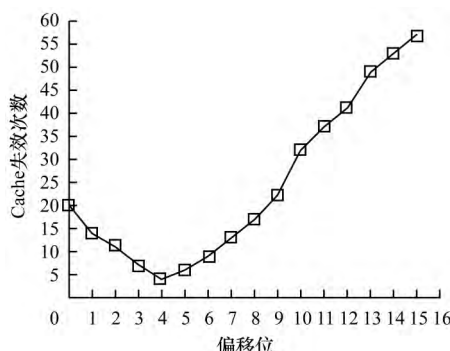


图7 Cache 失效次数

实验结果显示,偏移位为 4 的元素发生 Cache 失效的次数最少,其中由于噪声影响,在偏移位为 4 的位置也发生了 Cache 失效,不排除附近才是真正的偏移位,选取附近候选值的个数与获取密钥成功率的关系如表 1 所示。

表1 不同候选值个数下获取密钥的成功率 %

候选值个数	成功率
1	85.6
3	97.3
5	100.0

4 结束语

本文首先介绍了 DES 算法的实现原理,针对 Flush + Reload 方法单次攻击只能确定 S 盒行信息,无法确定使用 S 盒的行偏移位问题,提出一种基于 S 盒在 Cache 中会发生不对齐分布的特性来确定具体偏移位的方法,利用软件实现时 S 盒泄露的旁路信息获取密钥。相比传统 Cache 计时攻击方法,该攻击方法具有精准、高效的特点,并且不需要采集大量的旁路信息,能够有效解决使用穷搜等方法来确定密钥偏移位的问题。然而 Flush + Reload 方法依靠 cflush 指令,如果系统限制 cflush 指令则会造成攻击方法失效,因此,下一步将研究更具通用性的数据驱逐方法,使攻击更具有普遍性。

参考文献

- [1] PERCIVAL C. Cache missing for fun and profit [EB/OL]. [2017-10-10]. <http://www.daemonology.net/papers/htt.pdf>.
- [2] YAROM Y, FALKNER K. FLUSH + RELOAD: a high resolution, low noise, L3 cache side-channel attack [C] // Proceedings of USENIX Conference on Security Symposium. [S. l.]: USENIX Association, 2014: 719-732.
- [3] 赵新杰,王 韬,郑媛媛. Camellia 访问驱动 Cache 计时攻击研究 [J]. 计算机学报, 2010, 33(7): 1153-1164.
- [4] GULLASCH D, BANGERTER E, KRENN S. Cache games-bringing access-based cache attacks on AES to practice [C] // Proceedings of IEEE Symposium on Security and Privacy. Washington D. C., USA: IEEE Computer Society, 2011: 490-505.
- [5] GULMEZOGLU B, INCI M, IRAZOKI G, et al. Cross-VM cache attacks on AES [J]. IEEE Transactions on Multi-scale Computing Systems, 2017, 2(3): 211-222.
- [6] IRAZOQUI G, INCI M S, EISENBARTH T, et al. Wait a minute! a fast, cross-VM attack on AES [J]. Lecture Notes in Computer Science, 2014, 8688: 299-319.
- [7] IRAZOQUI G, INCI M S, EISENBARTH T, et al. Know thy neighbor: crypto library detection in cloud [J]. Proceedings on Privacy Enhancing Technologies, 2015(1): 25-40.
- [8] GRUSS D, SPREITZER R, MANGARD S. Cache template attacks: automating attacks on inclusive last-level caches [C] // Proceedings of USENIX Conference on Security Symposium. [S. l.]: USENIX Association, 2015: 897-912.
- [9] 解双建,原 亮,谢方方. DES 算法原理及其 FPGA 实现 [J]. 计算机技术与发展, 2011, 21(7): 158-160.
- [10] PAAR C, PELZL J. 深入浅出密码学: 常用加密技术原理与应用 [M]. 马小婷, 译. 北京: 清华大学出版社, 2012.
- [11] 李 超,孙 兵,李瑞林. 分组密码的攻击方法与实例分析 [M]. 北京: 科学出版社, 2010.
- [12] 赵新杰,王 韬,郭世泽,等. AES 访问驱动 Cache 计时攻击 [J]. 软件学报, 2011, 22(3): 572-591.
- [13] 陈财森,王 韬,郭世泽,等. 针对 RSA 算法的踪迹驱动数据 Cache 计时攻击研究 [J]. 计算机学报, 2014, 37(5): 1039-1051.
- [14] XUAN C, COPELAND J A, BEYAH R A, et al. Toward revealing kernel malware behavior in virtual execution environments [J] // Proceedings of International Symposium on Recent Advances in Intrusion Detection. Saint-Malo, France: [s. n.], 2009: 304-325.
- [15] LIPP M, GRUSS D, SPREITZER R, et al. ARMageddon: last-level cache attacks on mobile devices [J]. USENIX Security Symposium, 2015, 6(1): 549-569.

编辑 金胡考